RAG Architecture for Student Learning Platforms

https://gemini.google.com/app/dd6ccf4b07dc8475

User prompt: my ultimate goal is to build a platform for students where they can learn curated, llm generated notes, exams for the pdfs, slides they upload on the platform. the ocr part is handled, i am researching on what could be better representation such as knowledge graph or other representation so that i can have the information stored efficiently such that i dont have to pass the whole pdf text to llm as input. what are techniques to store, retrieve and inject information

Response: To build an efficient learning platform for student notes and exam generation without overloading the LLM context window, you should move beyond flat chunk-and-embed vector storage. Academic slides and PDFs have strong visual and structural hierarchy, conceptual dependencies, and distinct granularities (e.g., high-level lecture themes vs. precise definitions).

Here are the primary representations, retrieval methods, and injection techniques engineered for educational content.

1. Data Representation Strategies

Representation Type	How It Works	Primary Use Case for Education
Hierarchical Trees (Parent-Child Indexing)	Splits text into small leaf chunks (100–200 tokens) linked to larger parent sections or slide summaries (500–1000 tokens).	High-level note generation without losing original detailed context.
Knowledge Graphs (GraphRAG)	Extracts key concepts as nodes and conceptual dependencies as edges (`PREREQUISITE_OF`, `CONTRASTS_WITH`, `EXAMPLE_OF`).	Generating structured study guides, concept maps, and cross-topic exam questions.
Tree-Organized Summaries (RAPTOR)	Recursively clusters and summarizes document chunks from the bottom up to build a multi-resolution tree.	Global synthesis questions (e.g., "Summarize Chapter 3") vs. precise lookup questions.
Structured Metadata Tables	Extracts metadata like `lecture_number`, `heading_level`, `slide_number`, `has_formula`, `is_definition`.	Filtering context during exam generation (e.g., retrieve only formulas from Lecture 4).

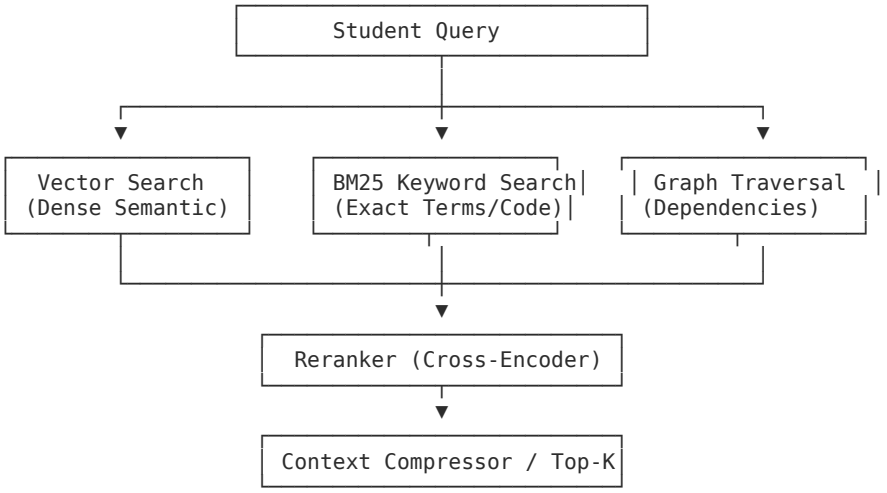
Knowledge Graph vs. Hierarchical Trees

- **Knowledge Graphs:** Best for **Exam Generation**. You can traverse 2-hop subgraphs to create multi-hop reasoning questions (e.g., asking how *Concept A* impacts *Concept C* through intermediate *Concept B*).
- **Hierarchical Trees:** Best for **Notes & Summaries**. They preserve the flow of the professor's slides and syllabus order, preventing the LLM from generating out-of-order study guides.

2. Storage & Retrieval Techniques

Hybrid Search Architecture

To avoid passing entire PDFs, combine three distinct retrieval paths in a single hybrid pipeline:



1. **Dense Vector Search (Semantic):** Finds semantically similar passages (e.g., searching for "cell energy production" matches "mitochondria ATP synthesis").
2. **Sparse Keyword Search (BM25):** Ensures exact matches for technical terms, formulas, variable names, and acronyms that dense embeddings often dilute.
3. **Graph Subgraph Retrieval:** Fetches related concepts and definitions linked to the primary target node.

4. **Cross-Encoder Reranking:** Passes the top ~20 retrieved candidate chunks through a reranker model (such as BGE-Reranker or Cohere Rerank) to trim the context to the top 3–5 most relevant items before calling the LLM.

3. Context Injection Techniques

When building notes or exams, **how** you format the retrieved information inside the prompt dictates output quality and token consumption:

A. Small-to-Big (Parent-Child) Expansion

- **Technique:** Perform vector search against small leaf chunks (e.g., a single slide bullet point) for exact matching accuracy. When injecting into the LLM, swap the leaf chunk with its **Parent Section Summary** or surrounding page context.
- **Benefit:** High search precision without sending disconnected, contextless snippets to the LLM.

B. Structured Context Serialization (YAML/JSON Injection)

Instead of feeding raw text paragraphs to the LLM, inject the extracted context in a structured format:

```
context:
  topic: "Neural Networks: Backpropagation"
  slide_source: "Lecture 5, Slide 12"
  prerequisite_concepts:
    - "Chain Rule"
    - "Loss Functions"
  key_formulas:
    - "\partial E / \partial w"
  core_definitions:
    - "Gradient Descent: Iterative optimization algorithm..."
```

- **Benefit:** Structured representations force the LLM to adhere strictly to source facts, reducing hallucinations and allowing targeted exam item generation across specific cognitive domains (Bloom's Taxonomy).

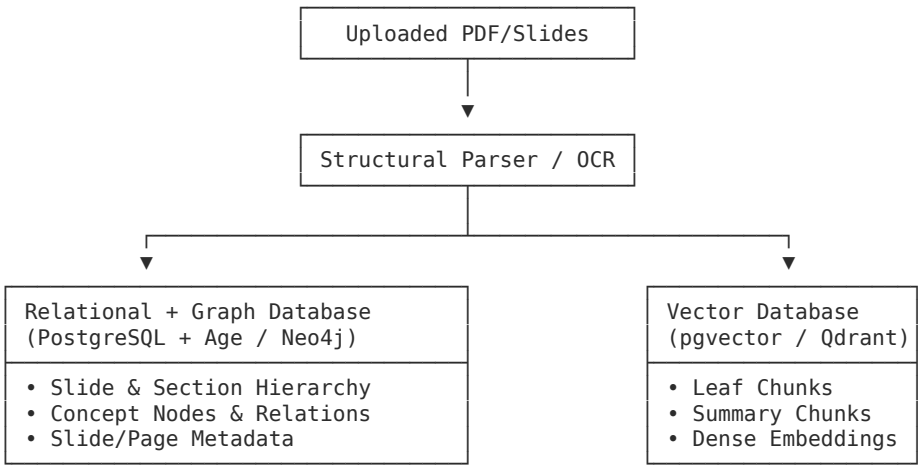
C. Map-Reduce for Whole-Deck Notes (Community Summarization)

When a student requests comprehensive study notes for a 100-slide deck:

1. Partition the knowledge graph or document tree into logical clusters/communities (e.g., using the Leiden algorithm or document chapter boundaries).
2. Run a **Map phase**: Generate concise study notes for each chapter/cluster independently in parallel.
3. Run a **Reduce phase**: Combine the structured chapter notes into an overall master study guide.

4. Recommended System Architecture

To implement this efficiently, structure your backend storage into a dual-database layout:



End-to-End Workflow for Exam Generation

1. **Student Request:** "Generate a 5-question quiz on Module 2 with a focus on problem-solving."
2. **Metadata Filter:** Query the DB for all concept nodes under `Module 2`.
3. **Graph Traversal:** Select nodes with high node centrality (core concepts) and their immediate 1-hop dependencies.

4. **Targeted Retrieval:** Retrieve the corresponding leaf formulas and slide text for those specific nodes.
5. **Prompt Injection:** Construct a payload with target concepts, formulas, and distractor topics (derived from adjacent graph nodes).
6. **LLM Generation:** LLM outputs multiple-choice or short-answer questions with source slide citations (e.g., *"Source: Slide 14"*).

Sources

1. GraphRAG: Graph-Based Retrieval-Augmented Generation

User prompt: also when building notes/answers for users, we'd also want to add the exact images from slides/pdfs. we also need to be able to store those images so that we can paste them in the right context

Response: I encountered an error doing what you asked. Could you try again?

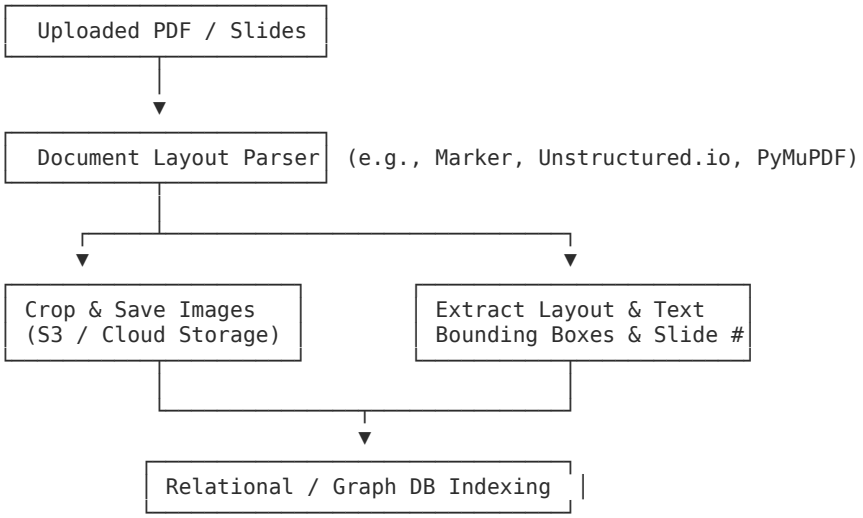
User prompt: also when building notes/answers for users, we'd also want to add the exact images from slides/pdfs. we also need to be able to store those images so that we can paste them in the right context

Response: To seamlessly incorporate exact diagrams, charts, and slide figures into your generated notes and study materials, you need a **multimodal extraction, indexing, and contextual rendering pipeline**.

Here is how to store, represent, retrieve, and inject visual content into your platform's LLM outputs.

1. Image Extraction & Storage Pipeline

When processing an uploaded PDF or slide deck, isolate visual assets into Object Storage while retaining their structural provenance in your primary database.



Storage Schema (`images` Table)

Store extracted images in object storage (AWS S3, GCP Cloud Storage) and save their metadata in your database:

- `image_id`: Unique Identifier (`img_slide05_fig02`).
- `s3_url`: Public/presigned URL to render in frontend Markdown.
- `document_id` / `slide_number`: Structural anchor.
- `bounding_box`: Location coordinates on the original slide.
- `vlm_caption`: LLM/Vision generated detailed description of the image content.

2. Multimodal Indexing & Representation

To know *when* an image is relevant to a user query or a section of generated notes, represent images using a **Dual-Indexing Strategy**:

A. Graph & Structural Linking (Explicit Context)

In your Knowledge Graph or Document Tree, attach the image node directly to its parent text chunk or slide section:

- `(Slide 5 Text Chunk)` --[CONTAINS_IMAGE]--> `(Image_123)`
- `(Concept: "Krebs Cycle")` --[ILLUSTRATED_BY]--> `(Image_123)`

When the LLM retrieves the text chunk for "Krebs Cycle," it automatically pulls along the associated `Image_123` metadata.

B. Dense Multimodal Embeddings (Implicit Context)

Generate visual embeddings for diagrams so users can search for visual concepts directly:

- **Option 1 (Text-based):** Pass extracted images through a Vision-Language Model (VLM like GPT-4o mini or SigLIP) to create a rich summary (e.g., *"Flowchart illustrating glycolysis steps from glucose to pyruvate with enzyme labels"*). Embed this summary text in your vector store.
- **Option 2 (Visual-based):** Use late-interaction vision models like **ColPali** or **SigLIP embeddings** to index visual PDF pages directly without relying solely on text OCR.

3. Retrieval & Selection Strategy

When a student requests study notes or an answer, run retrieval in two steps:

1. **Text Chunk & Graph Retrieval:** Retrieve the primary text context needed to answer the question or generate the topic note.
2. **Associated Image Fetching:**
    - **Primary Rule:** Fetch all images directly linked to the top-K retrieved text chunks or slide nodes via structural foreign keys.
    - **Fallback Rule (Semantic Search):** If the user explicitly asks for visual context (e.g., *"Show me a diagram of..."*), execute a vector search against the image caption index to find the top matching figure.

4. LLM Context Injection & Output Rendering

LLMs cannot output raw image pixels, but they excel at placing formatted Markdown tags (`![caption](url)`) inside generated text if you provide them with image metadata.

A. System Prompt & Context Payload

Inject candidate images into the prompt's context block using a structured format, explicitly giving the LLM permission to insert them where relevant.

```
context:
  text_chunks:
    - chunk_id: "chunk_99"
      content: "Mitosis consists of four primary phases: Prophase, Metaphase, Anaphase, and Telophase..."
  associated_images:
    - image_id: "img_slide_08_01"
      url: "https://cdn.yourplatform.com/docs/doc456/slide08_fig1.png"
      alt_text: "Diagram showing four stages of mitosis with labeled chromosomes"
      parent_chunk_id: "chunk_99"

instructions:
  - You are generating curated study notes.
  - If a retrieved image directly illustrates a section you are writing, insert it inline using standard Markd
  - Do NOT invent or fabricate image URLs. Only use URLs explicitly provided in the associated_images list.
```

B. Generated Output Example

The LLM will dynamically weave the image into the generated response right where it aids student comprehension:

> **Stages of Mitosis**
>
> Mitosis is the process of nuclear division in eukaryotic cells, divided into four key phases:
>
> 1. **Prophase:** Chromatin condenses into distinct chromosomes.
> 2. **Metaphase:** Chromosomes line up along the metaphase plate.
> 3. **Anaphase:** Sister chromatids are pulled to opposite poles.
> 4. **Telophase:** Nuclear membranes reform around separated chromosomes.
>
> *Figure 1: Visual breakdown of mitosis stages (Source: Lecture 3, Slide 8)*

Key Pitfalls & Best Practices

1. **Avoid Image Flooding:** Filter out non-educational images (university logos, background slide graphics, bullet-point icons) during the extraction phase using image aspect ratio, resolution thresholds, or a lightweight vision classifier.

2. **Token Efficiency:** Do not pass actual image bytes into the LLM during note generation unless necessary. Passing the generated **text caption + image URL** in the prompt saves massive token costs while yielding identical inline image placement results.

3. **Frontend Rendering:** Use a frontend Markdown parser (like `react-markdown` with `rehype-raw`) that supports zoomable lightboxes or modal previews when students click on the embedded slide figures.
